The Scrum Framework and DevOps

A Popular Agile Framework, and the Delivery Practices Around It

Agile vs Scrum

Scrum: A Framework for Practicing Agile Values

Agile = Values and Principles

What and Why

- Respond to change
- Customer collaboration
- Working software
- Individuals and interactions

Abstract principles - no specific "how"

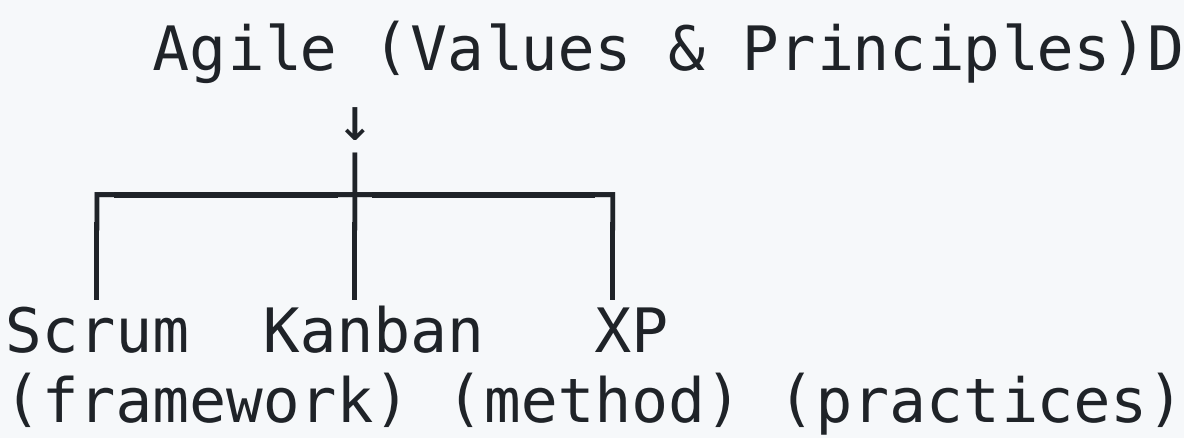
Scrum = A Lightweight Framework

One way to work in an Agile manner

- Sprints (one month or less, often 2 weeks)
- Daily Scrum (15 min)
- Sprint Planning / Review / Retrospective
- Accountabilities: Product Owner, Scrum Master, Developers

Concrete events and rules - a specific "how"

The Relationship



Scrum is one way to put Agile values into practice.

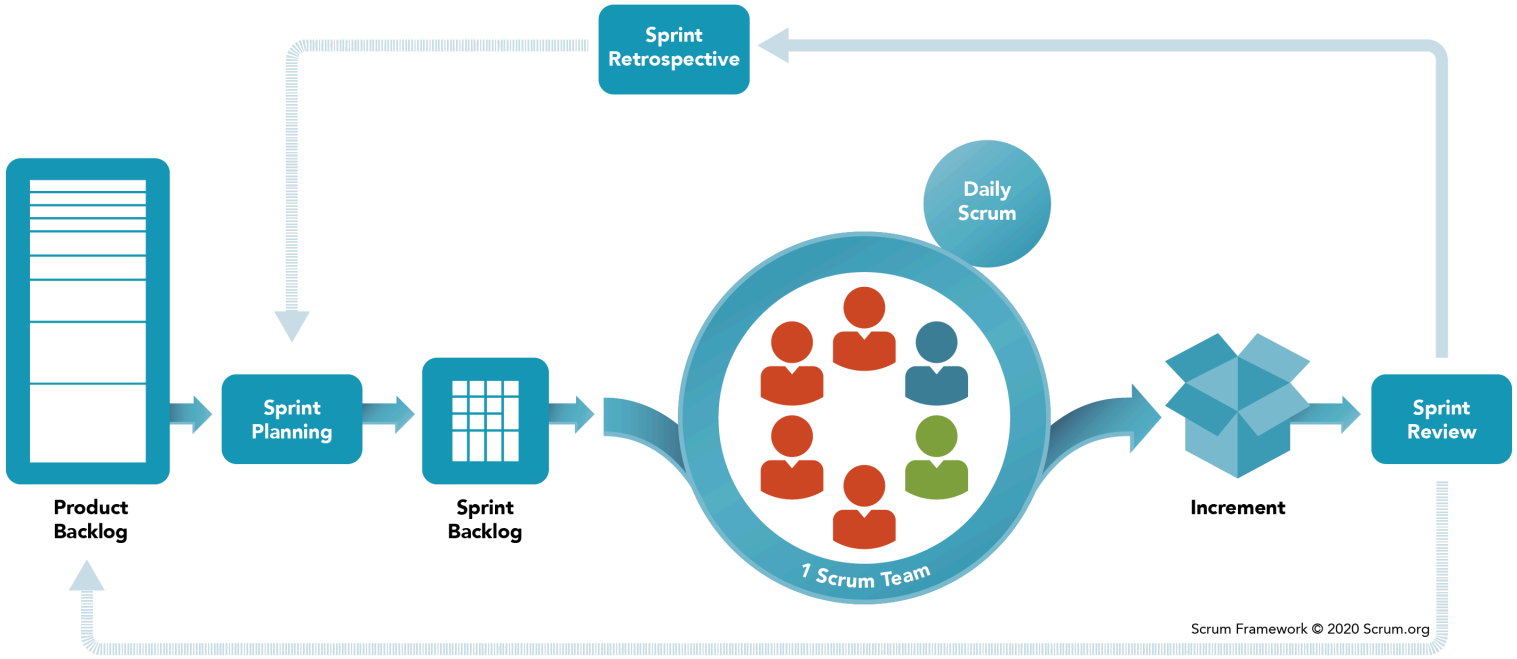
(Like Java is one language you can use to practice OOP.)

IMPORTANT

Caution: doing the Scrum events mechanically does **not** make a team Agile.

- The values matter more than the ceremony.
- Scrum practices can help teams put Agile principles into practice.

Scrum: One of the Most Widely Used Agile Frameworks



Surveys of Agile teams (e.g., the annual *State of Agile* reports) consistently report Scrum as the most common choice.

The Scrum Team: Three Accountabilities

The Scrum Guide (2020) describes **one Scrum Team** with three accountabilities, not a "Development Team" plus others.

Product Owner

- Accountable for maximizing the value of the product
- Accountable for effective Product Backlog management (may delegate the work)

Scrum Master

- Accountable for the Scrum Team's effectiveness
- Coaches the team, helps remove impediments, and makes sure the events are positive, productive, and timeboxed

Developers

- The people committed to creating any aspect of a usable Increment each Sprint
- Self-managing and cross-functional

Team size

- The whole Scrum Team is typically **10 people or fewer**

Stakeholders

- Users, clients, and others with an interest in the product
- They are **not** part of the Scrum Team, but they interact with it (e.g., at the Sprint Review)

Scrum Terminology

Scrum Values

Scrum works best when the team lives these five values:

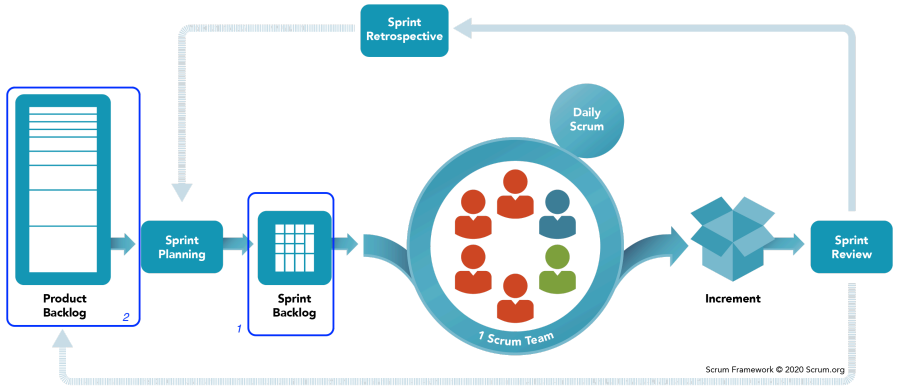
- **Commitment**: to the team's goals and to supporting each other
- **Focus**: on the work of the Sprint and the Sprint Goal

- **Openness**: about the work and the challenges
- **Respect**: for each other as capable, independent people
- **Courage**: to do the right thing and to work on tough problems

Scrum Backlogs

- **Product Backlog**: An evolving, **ordered** list of work to improve the product and achieve the **Product Goal**.
- **Sprint Backlog**: The **Sprint Goal**, selected Product Backlog Items, and the Developers' delivery plan.

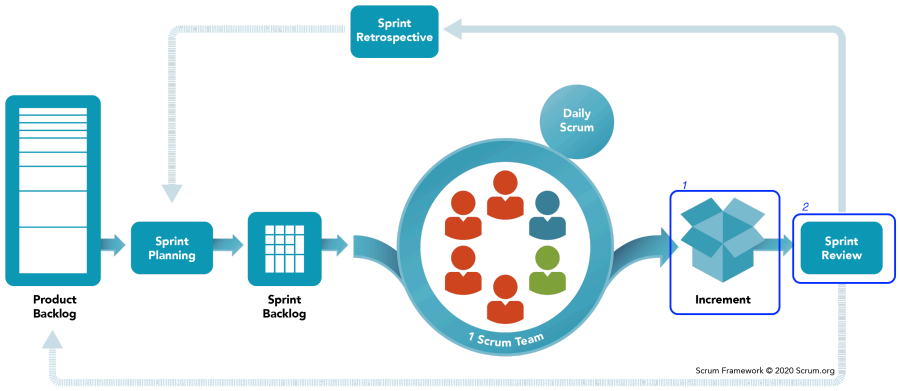
- The Product Backlog is **continuously refined**; items are selected during **Sprint Planning**.



> In simple terms, a backlog is a to-do list—but an ordered, evolving one.

Scrum Increment

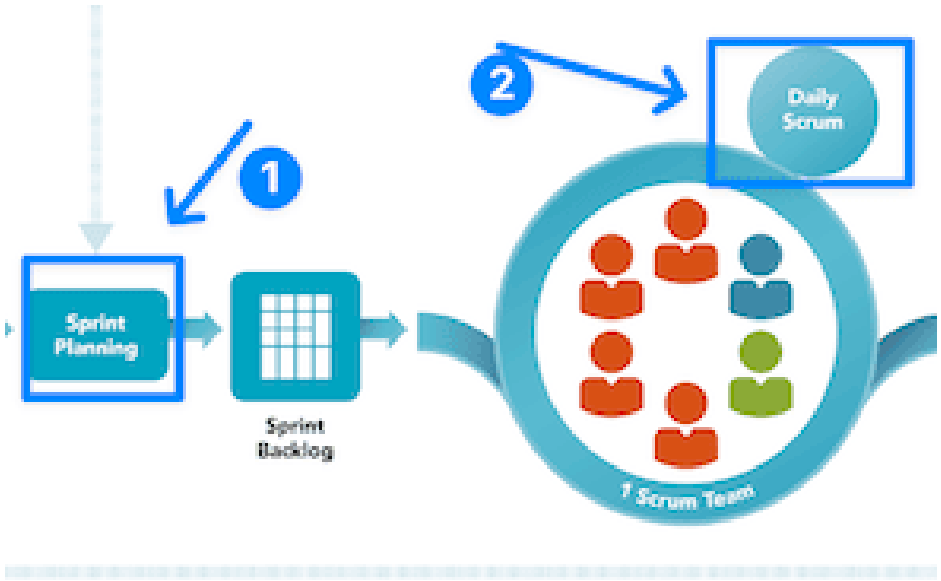
- **Increment**: A usable, **Done**-compliant step toward the Product Goal that works with prior Increments.
- A Sprint may produce multiple Increments.
- Refine the Product Backlog as needed, including based on the Increment and Sprint Review feedback.



> In this course, an Increment is a usable product improvement represented by code, tests, and documentation in the GitHub repository.

Sprint Planning & Daily Scrum

1. **Sprint Planning** (Start of sprint)
    - Why is this Sprint valuable? (Sprint Goal)
    - What can be done this Sprint?
    - How will the chosen work get done?
2. **Daily Scrum** (Every day, 15 minutes)
    - Purpose: inspect progress toward the Sprint Goal and adapt the Sprint Backlog
    - Developers choose the format



> In this and other ASE courses, we do `weekly`, not `daily`, scrum.

Daily Scrum: The Three Questions Are Optional

A popular way to run it:

- What did I do yesterday?
- What will I do today?
- Any blockers?

This is a common technique, **not** a required format. Many teams instead walk through the Sprint Backlog board and ask: "What is blocking the Sprint Goal?"

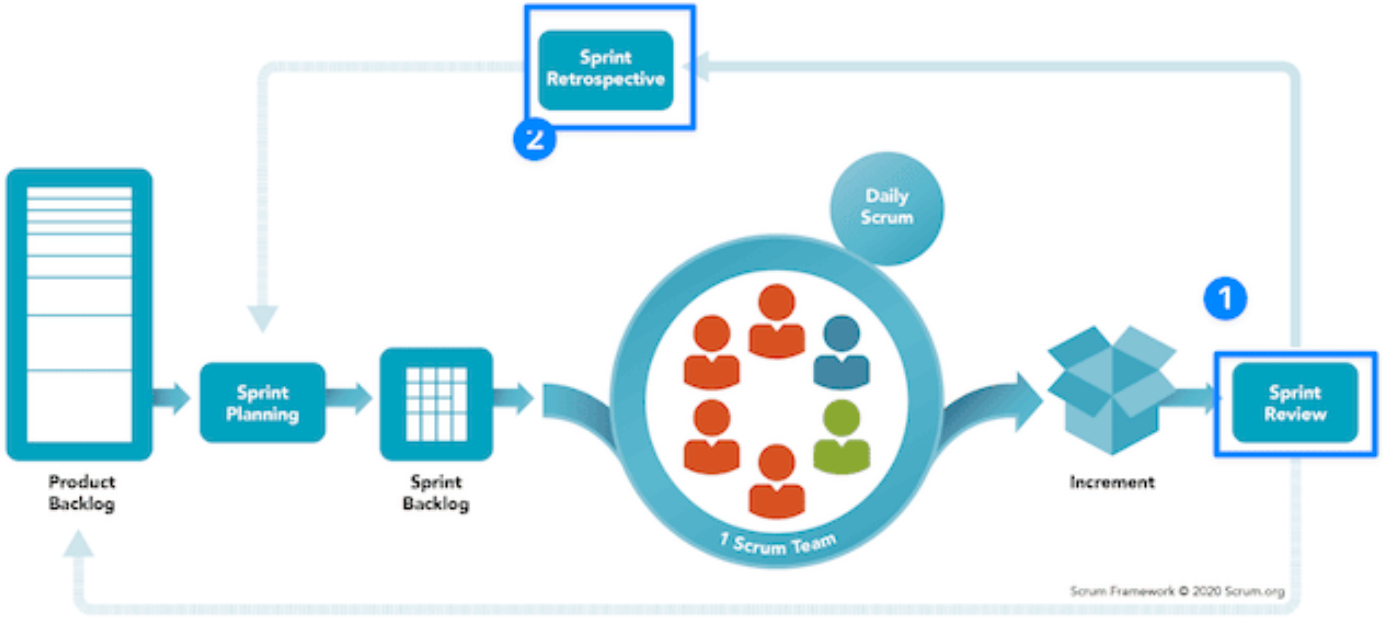
Sprint Review & Retrospective at the end of the Sprint

Sprint Review

- Present the results to stakeholders (more than a demo)
- Discuss progress toward the Product Goal
- Decide together what to adapt next

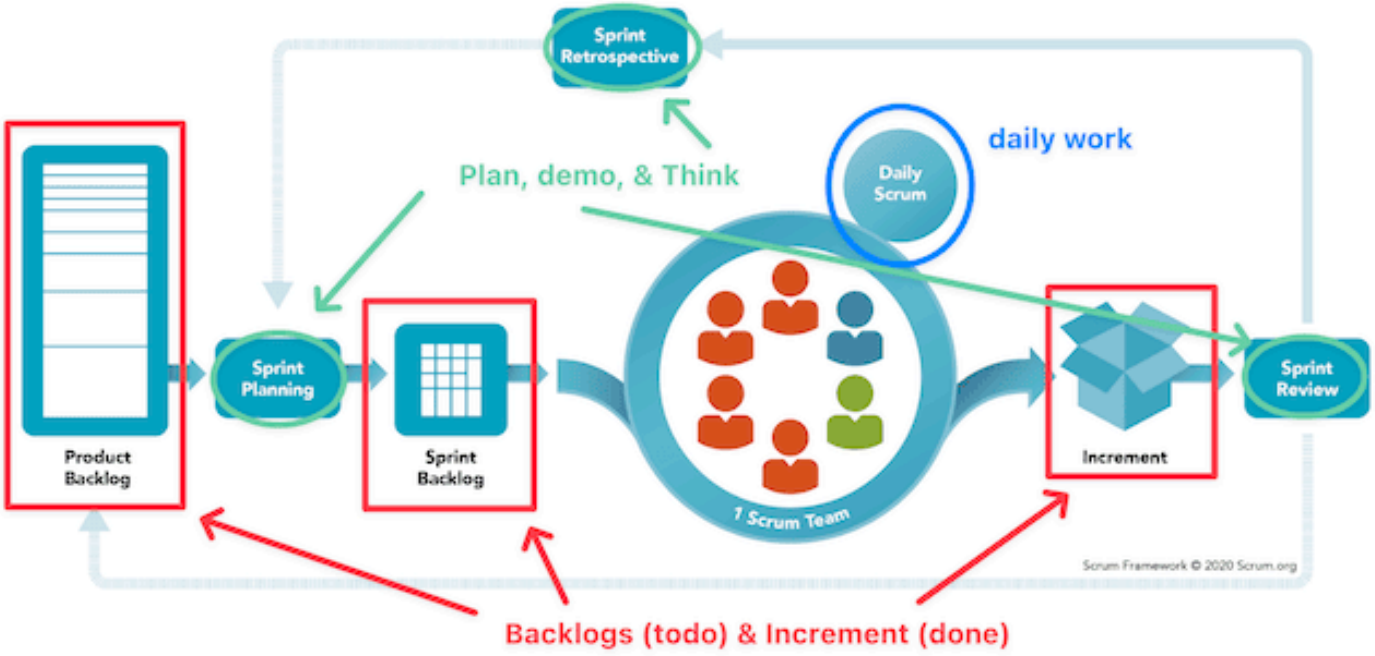
Sprint Retrospective

- What went well? What can improve?
- Plan ways to increase quality and effectiveness


Sprint Retrospective
2
Daily Scrum
1
Product Backlog
Sprint Planning
Sprint Backlog
1 Scrum Team
Increment
Sprint Review
Scrum Framework © 2020 Scrum.org

Overall Perspective of Scrum

- We plan, review (inspect the Increment), and reflect (Retrospective) every Sprint.
- We maintain a to-do list (Product Backlog, with a Sprint Backlog for the current Sprint) and a done list (the Increment).
- The Product Backlog is refined **continuously**. The Developers adjust the Sprint Backlog **every day** as they learn.


Sprint Retrospective
Plan, demo, & Think
Daily Scrum
daily work
Sprint Planning
Product Backlog
Sprint Backlog
1 Scrum Team
Increment
Sprint Review
Scrum Framework © 2020 Scrum.org
Backlogs (todo) & Increment (done)

Sprints from a Software Engineer's Perspective

- Use the Sprint Backlog to guide the work.
- Build, test, and deploy usable Increments.
- Review Increments with stakeholders; update the Product Backlog as needed.
- Choose process improvements in the Retrospective and add them to the next Sprint Backlog.


Agile
Define
Design
Deploy
Test
Develop
SPRINT'S OUTCOME
SPRINT'S CUMULATIVE OUTCOME
SPRINT'S CUMULATIVE OUTCOME

Real-World Cases

Famous cases are useful for discussion, but ask: **What was due to Scrum, and what was due to other things?**

Typical contributing factors besides Scrum:

- Architecture (e.g., microservices)
- Organization design (small, autonomous teams)
- Automation (CI/CD, monitoring)
- Culture and leadership

1. Spotify

- Around 2012, Henrik Kniberg and Anders Ivarsson described how Spotify organized its engineering teams: **Squads**, **Tribes**, **Chapters**, **Guilds**.
- It was a snapshot of an evolving, experimental way of working, **not a framework to copy**.
- Spotify's own people later stressed that teams adapted it continuously.

Discussion: If you copy the names (squads, tribes), do you get the same results?

2. Amazon

- Moved from a large monolithic application toward **services** owned by small teams ("two-pizza teams").
- Strong emphasis on automated, frequent deployment.
- Quote: "You build it, you run it" - Werner Vogels, Amazon CTO (2006 interview)

Discussion: This is mainly a story about **architecture, ownership, and operations automation** (DevOps), not only about Scrum. Why?

Agile vs Traditional: Be Skeptical of Statistics

You will often see tables like "Agile projects succeed 2x more often."

Before using such numbers, ask:

- **Who** ran the survey, and how many were asked?
- How were "success", "on time", and "Agile" **defined**?
- Do the compared projects have similar size, risk, and context?
- Is it **correlation** or **causation**?

Exercise: Find one such statistic online. Trace it back to its original source. What does it actually say?

What Spiral Taught Us, What Agile Added

1. Ideas that Agile builds on (from Spiral and other iterative models):

- Iterative development
- Risk awareness
- Prototyping
- Customer involvement

2. What Agile methods emphasize:

- Short iterations (typically weeks)
- Self-organizing teams
- Technical excellence practices
- Embracing change
- Frequent delivery
- Individuals and interactions

Agile does **not** abandon planning, documentation, or risk management. It does them "just enough", in smaller steps.

Why Both Still Matter

1. When Plan-Driven (Waterfall/Spiral-style) Elements Help:

- High-risk or safety-critical systems (aerospace, medical devices)
- Regulatory requirements and traceability
- Hardware-software integration
- Fixed-scope contracts with stable requirements

(Many regulated teams also use iterative and Agile practices inside these constraints.)

2. When Agile Practices Help:

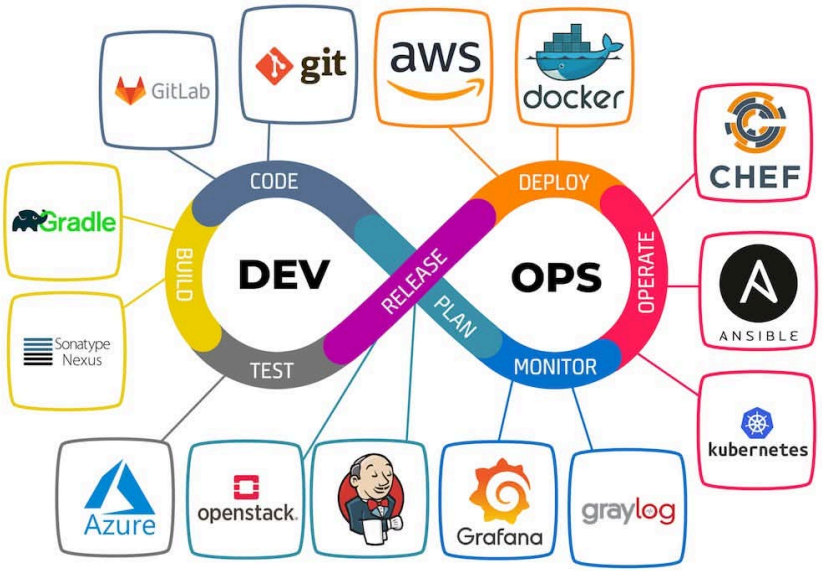
- Uncertain or fast-changing requirements
- Need for fast feedback
- Innovation and experimentation
- Products that evolve continuously

Modern Evolution: DevOps (2009+)

Agile practices often stopped at the boundary of the development team, but...

- Operations were often still a separate group
- Deployment was often manual and risky
- "Throw it over the wall" mentality

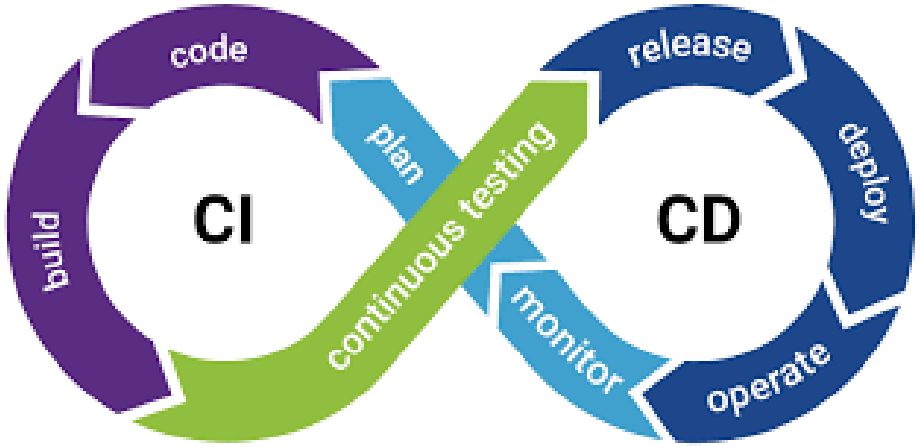
DevOps: a culture and set of practices that bring development, operations (and often security and quality) together



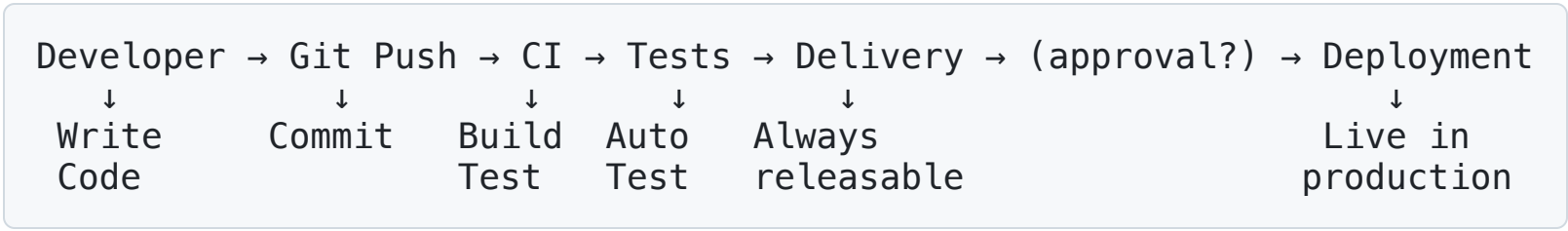
- Shared responsibility for the running product
- Automation of build, test, and release
- Fast feedback from production (monitoring)
- Reliable, frequent delivery

DevOps is not just "Agile extended to operations". It is about collaboration, automation, and feedback across the whole delivery chain.

CI/CD Pipeline



CI, Continuous Delivery, and Continuous Deployment



CI (Continuous Integration): Merge code frequently, build and test automatically

Continuous Delivery: Keep the software **always in a releasable state**; releasing is a (business) decision, often one click

Continuous Deployment: Every change that passes the automated checks is **deployed to production automatically**

"CD" can mean either of the last two. Always ask which one.

Infrastructure as Code

Treat servers like code

Traditional Way (Manual): ❌ Hard to reproduce, error-prone

```
1. Click AWS console
2. Create server manually
3. Install software by hand
4. Configure settings
```

DevOps Way (Code): ✅ Run script → Infrastructure created!

Infrastructure as Code (IaC) (Write & Run):

- Describe infrastructure (servers, networks) in files
- Store the files in version control
- Run a tool to create or update it
- Repeatable and reviewable

Real Example 1: Terraform for AWS

```
# main.tf
resource "aws_instance" "web" {
  ami           = "ami-12345"      # Placeholder image ID
  instance_type = "t2.micro"       # Server size

  tags = {
    Name = "WebServer"
  }
}
```

Run: `terraform apply`

Result: An AWS server is provisioned (with a real AMI ID and credentials)

Real Example 2: Docker Compose

```yaml
# docker-compose.yml
services:
  web:
    image: nginx
    ports:
      - "80:80"

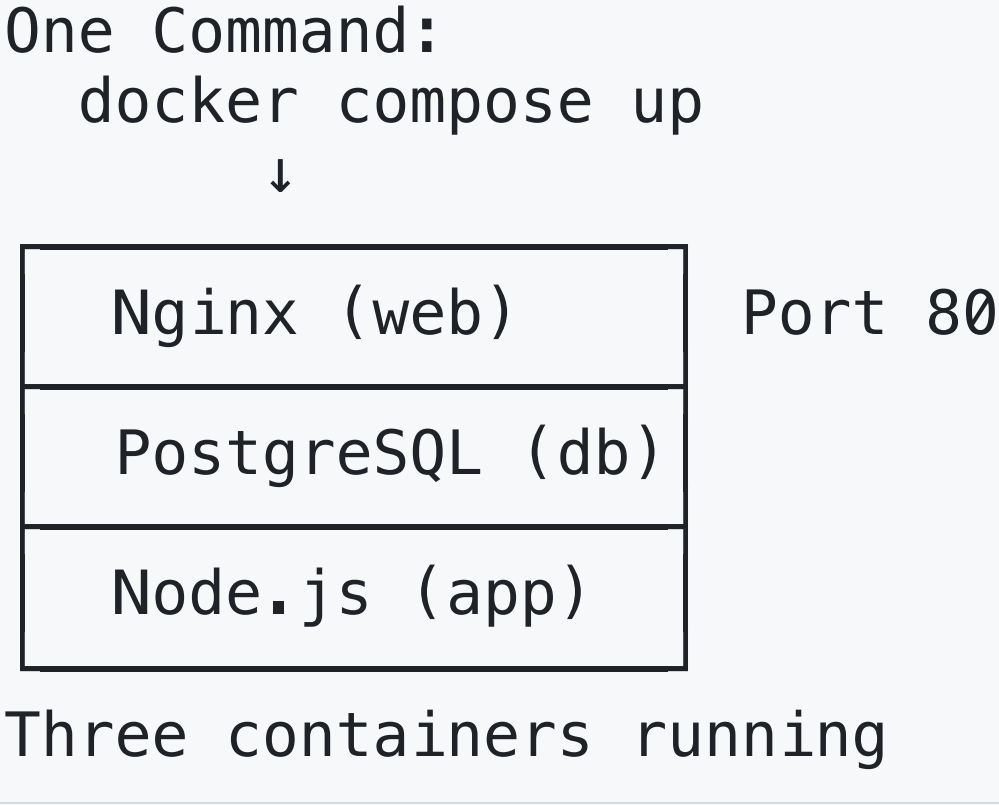
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example   # demo only; use secrets in real projects

  app:
    image: node:lts
    working_dir: /app
    volumes:
      - ./code:/app
    command: node index.js
```

Run: `docker compose up`

Result: Web server + Database + App containers running on your machine

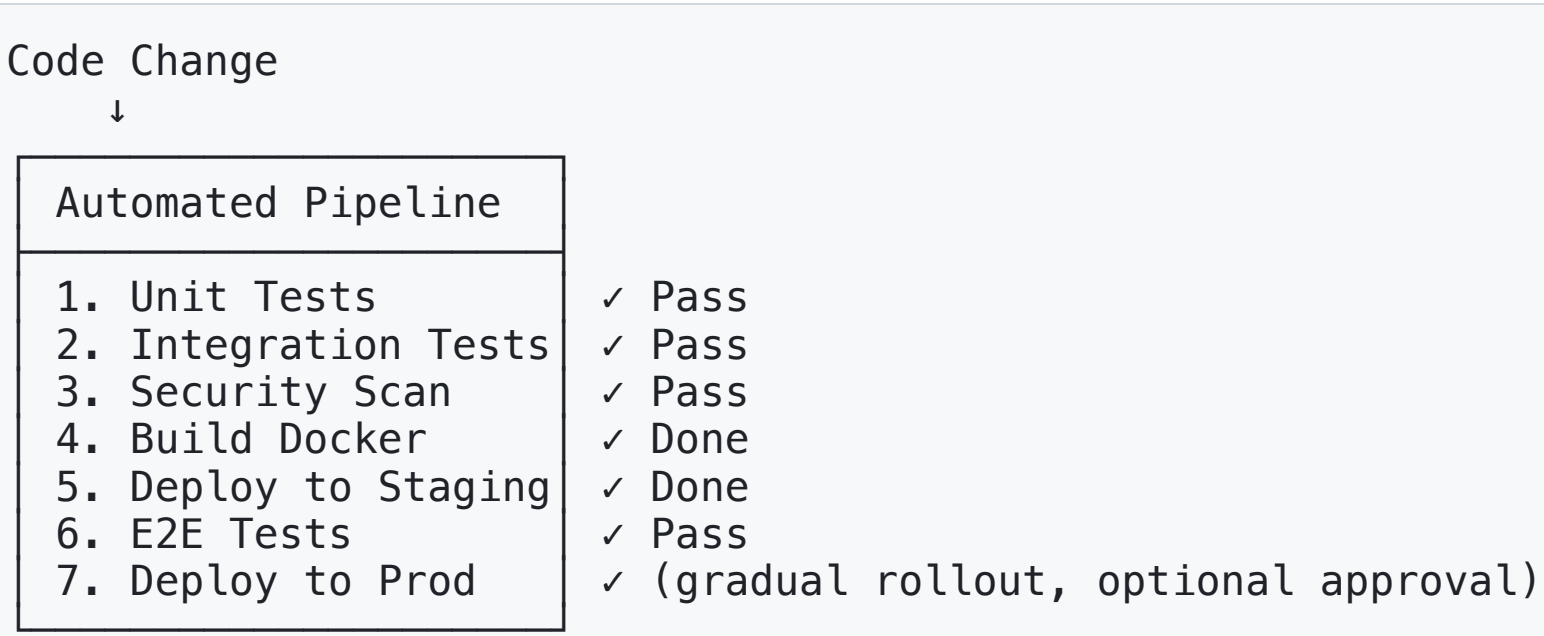
What This Does



Benefit: Share this file → Anyone can start the same multi-container environment

Automated Testing & Deployment

Automate repetitive, error-prone steps

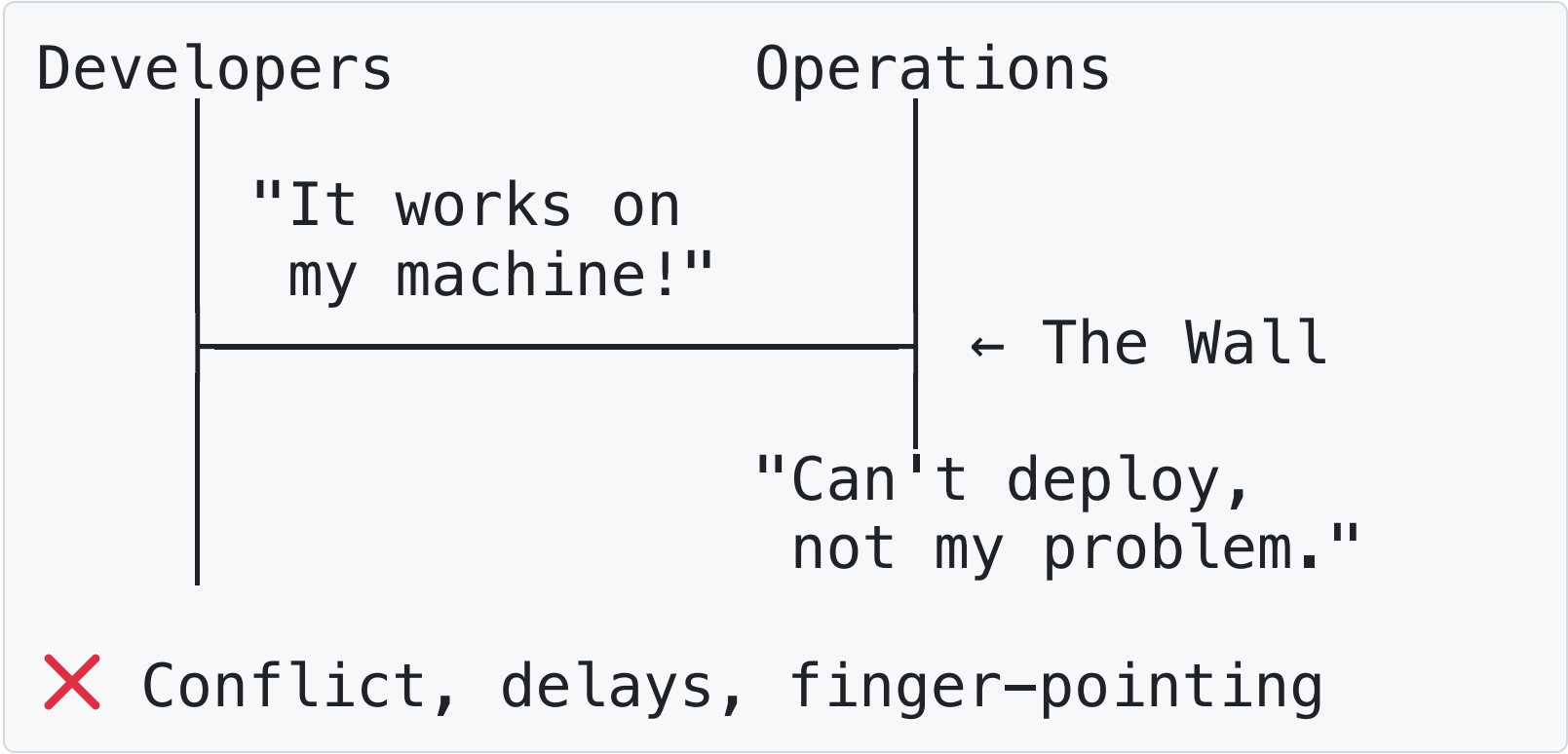


Traditional: Days of manual, error-prone work
With automation: Repeatable checks in minutes to hours

Humans are still essential: code review, approvals, gradual rollout, monitoring, rollback, and incident response.

Dev & Ops Collaborate

Before DevOps:



With DevOps:

```
   Dev + Ops Team
          |
    Shared goals
    Shared tools
    Shared responsibility

✅ Collaboration, faster and safer delivery
```

Result: "You build it, you run it" mentality

Docker vs Traditional

Traditional	Docker Compose
Install Nginx manually	`image: nginx`
Install PostgreSQL manually	`image: postgres:16`
Configure ports manually	`ports: "80:80"`
Often hours to set up	Often minutes once images exist

Docker packages applications into containers.

Compose describes a multi-container setup in a file.

Terraform-like IaC tools provision infrastructure itself (servers, networks). Containers and IaC are related, but not the same thing.

The Software Process Evolution Timeline

```
1970: Royce's paper (sequential model, later called "Waterfall")
       ↓ (Risky when requirements change and feedback is late)
1986–88: Spiral Model
       ↓ (Risk-driven iteration; often heavy in practice)
1991–1999: Many iterative / lightweight methods
       ↓ (RAD, Scrum, XP, FDD ...)
2001: Agile Manifesto
       ↓ (Shared values and principles)
2009: DevOps
       ↓ (Development + operations collaboration)
2020s: AI-Assisted Development
       (Next evolution?)
```

Approaches overlap and coexist. New ones do not simply replace the old ones.

Lessons We Learned in Software Process

1. Processes Evolve for Reasons

- Each model solved real problems
- Each created new problems
- Understanding history helps choose the right approach

2. No Silver Bullet

- Plan-driven approaches: good when requirements are stable or regulation demands traceability
- Spiral-style approaches: good for managing high-risk projects
- Agile approaches: good when uncertainty is high and feedback is valuable
- Choose based on context, and combine when needed!

3. People Matter as Much as Process

- A good process with the wrong team can still fail
- A simple process with a great team often succeeds
- Agile puts this idea at the center

4. Technical Practices Matter

- Automation makes frequent, safe delivery much easier
- Testing, CI/CD, refactoring enable speed

Conclusion: Match the Process to the Problem

The goal is not "Waterfall vs Agile". It is to **design the feedback cycle and level of control** to fit:

- Product and its **uncertainty**
- **Risk** and safety needs
- **Regulation** and contracts
- Organization and team **capability**

Discussion: For (1) a pacemaker's firmware, (2) a student-club mobile app, and (3) a government payroll system, which practices would you take from

Key Principles to Remember

From Spiral and other iterative models:

- Manage risk systematically
- Build prototypes to validate
- Get feedback early and often

From Agile:

- Deliver working software frequently
- Welcome changing requirements
- Empower self-organizing teams
- Reflect and adjust regularly

From DevOps:

- Automate repetitive work
- Share responsibility for the running product
- Learn quickly from production feedback